Challenge 05: Loadme

Category: Reversing Points: 30 Status: SOLVED

Flag

```
MOBISEC{dynamic_code_loading_can_make_everything_tricky_eh?}
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

Reverse-engineered a 3-stage dynamically-loaded APK: stage 1 XOR-decrypted with the package name, stage 2 XOR-decrypted with a fixed string key, final stage contained the hardcoded flag.

Each loading stage XOR-decrypts the next stage using a key derived from data already available on-device (the package name).

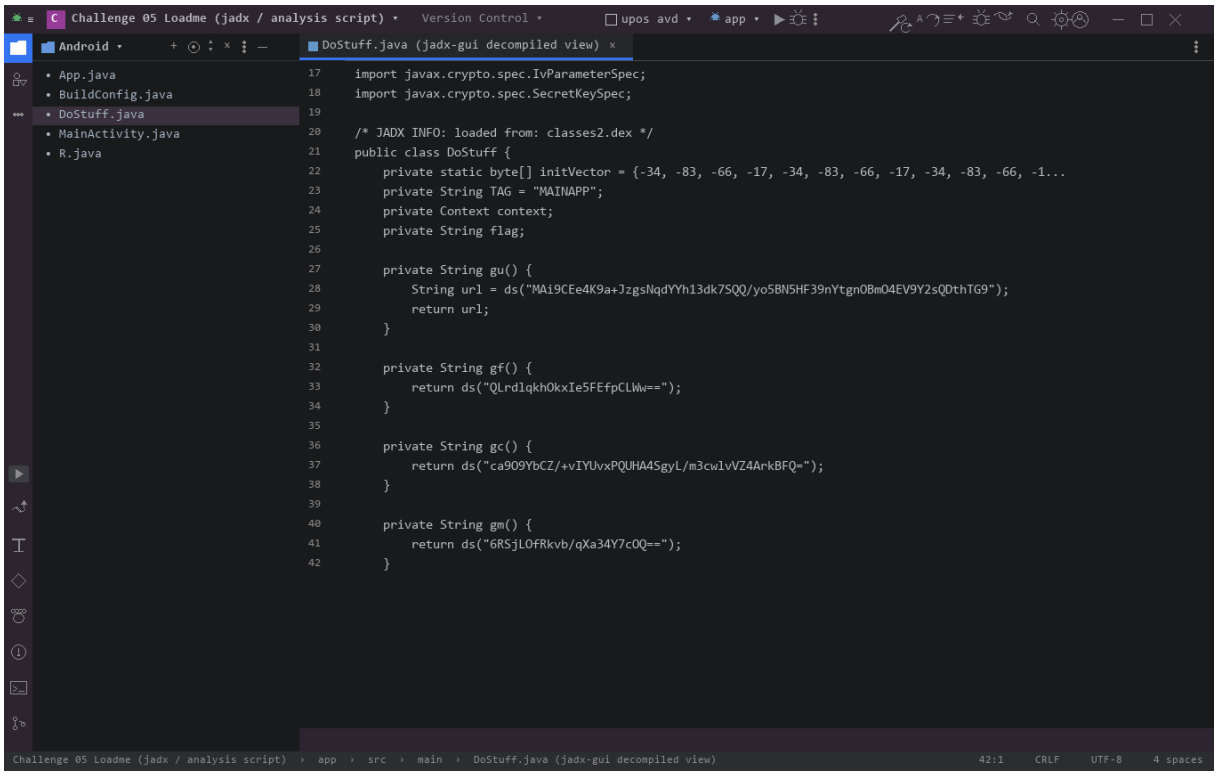
How I Got The Flag

1. I decompiled the APK and found it dynamically loads a 3-stage payload.

2. I identified that stage 1 is XOR-decrypted using the app's own package name as the key, and stage 2 using a fixed string key.

3. I wrote a script to reproduce both decryption steps and extracted the final stage, which contained the hardcoded flag.

Project Layout



Exact Code (from disk)

DoStuff.java (jadx decompiled)

```
package com.mobisec.dexclassloader;
import android.content.Context;
import android.os.StrictMode;
import android.util.Base64;
import dalvik.system.DexClassLoader;
import java.io.BufferedInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.lang.reflect.Method;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.regex.Pattern;
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
/* JADX INFO: loaded from: classes2.dex */
public class DoStuff {
    private static byte[] initVector = {-34, -83, -66, -17, -34, -83, -66, -17, -34, -83, -66, -17, -34, -83, -66, -17};
    private String TAG = "MAINAPP";
    private Context context;
    private String flag;
    private String gu() {
        String url = ds("MAi9CEe4K9a+JzgsNqdYYh13dk7SQQ/yo5BN5HF39nYtgnOBmO4EV9Y2sQDthTG9");
        return url;
    }
    private String gf() {
        return ds("QLrdlqkhOkxIe5FEfpCLWw==");
    }
    private String gc() {
        return ds("ca9O9YbCZ/+vIYUvxPQUHA4SgyL/m3cwlvVZ4ArkBFQ=");
    }
    private String gm() {
        return ds("6RSjLOfRkvb/qXa34Y7cOQ==");
    }
    private void setUserInput(String _flag) {
        this.flag = _flag;
    }
    private void setContext(Context ctx) {
        this.context = ctx;
    }
    private String dc(String url) {
        try {
            URL downloaded_url = new URL(url);
            HttpURLConnection urlConnection = (HttpURLConnection) downloaded_url.openConnection();
            urlConnection.connect();
            File file = new File(this.context.getCodeCacheDir(), gf());
            FileOutputStream fileOutput = new FileOutputStream(file);
            InputStream inputStream = urlConnection.getInputStream();
            byte[] buffer = new byte[1024];
            while (true) {
                int bufferLength = inputStream.read(buffer);
                if (bufferLength <= 0) {
                    fileOutput.close();
                    return file.getAbsolutePath();
```

```
            }
            fileOutput.write(buffer, 0, bufferLength);
        }
    } catch (Exception e) {
        return null;
    }
}
private boolean lc(String path) {
    File tmpDir = new File(this.context.getFilesDir().getAbsolutePath());
    File file = new File(path);
    DexClassLoader classloader = new DexClassLoader(file.getAbsolutePath(), tmpDir.getAbsolutePath(), null, ClassLoader.get
    file.delete();
    File[] ftemp = tmpDir.listFiles();
    for (File f : ftemp) {
        f.delete();
    }
    try {
        Class<?> classToLoad = classloader.loadClass(gc());
        Method method = classToLoad.getDeclaredMethod(gm(), Context.class, String.class);
        boolean res = ((Boolean) method.invoke(classToLoad, this.context, this.flag)).booleanValue();
        return res;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
public boolean start(Context ctx, String flag) {
    StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder().permitAll().build();
    StrictMode.setThreadPolicy(policy);
    setContext(ctx);
    setUserInput(flag);
    String path = dc(gu());
    da(path);
    return lc(path);
}
private void da(String path) {
    byte[] xorKey = this.context.getPackageName().getBytes();
    File file = new File(path);
    int size = (int) file.length();
    byte[] bytes = new byte[size];
    byte[] decbytes = new byte[size];
    try {
        BufferedInputStream buf = new BufferedInputStream(new FileInputStream(file));
        buf.read(bytes, 0, bytes.length);
        buf.close();
        for (int i = 0; i < size; i++) {
            decbytes[i] = (byte) (bytes[i] ^ xorKey[i % xorKey.length]);
        }
        File outFile = new File(path);
        FileOutputStream out = new FileOutputStream(outFile, false);
        out.write(decbytes);
        out.flush();
        out.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
private String ds(String enc) {
    try {
        String[] parts = this.context.getPackageName().split(Pattern.quote("."));
        String key = parts[1] + parts[0] + "key!!!";
```

```
            IvParameterSpec iv = new IvParameterSpec(initVector);
            SecretKeySpec skeySpec = new SecretKeySpec(key.getBytes("UTF-8"), "AES");
            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5PADDING");
            cipher.init(2, skeySpec, iv);
            byte[] original = cipher.doFinal(Base64.decode(enc.getBytes(), 0));
            return new String(original);
        } catch (Exception ex) {
            ex.printStackTrace();
            return null;
        }
    }
}
```

decode.py

```
import base64
from Crypto.Cipher import AES

pkg = "com.mobisec.dexclassloader"
parts = pkg.split(".")
key = (parts[1] + parts[0] + "key!!!").encode("utf-8")
iv = bytes([0xDE,0xAD,0xBE,0xEF]*4)
print("key", key, len(key))

def ds(enc_b64):
    raw = base64.b64decode(enc_b64)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    pt = cipher.decrypt(raw)
    padlen = pt[-1]
    pt = pt[:-padlen]
    return pt.decode()

vals = {
 "gu (url)": "MAi9CEe4K9a+JzgsNqdYYh13dk7SQQ/yo5BN5HF39nYtgnOBmO4EV9Y2sQDthTG9",
 "gf (filename)": "QLrdlqkhOkxIe5FEfpCLWw==",
 "gc (classname)": "ca9O9YbCZ/+vIYUvxPQUHA4SgyL/m3cwlvVZ4ArkBFQ=",
 "gm (methodname)": "6RSjLOfRkvb/qXa34Y7cOQ==",
}
for k,v in vals.items():
    print(k, "=>", ds(v))
```